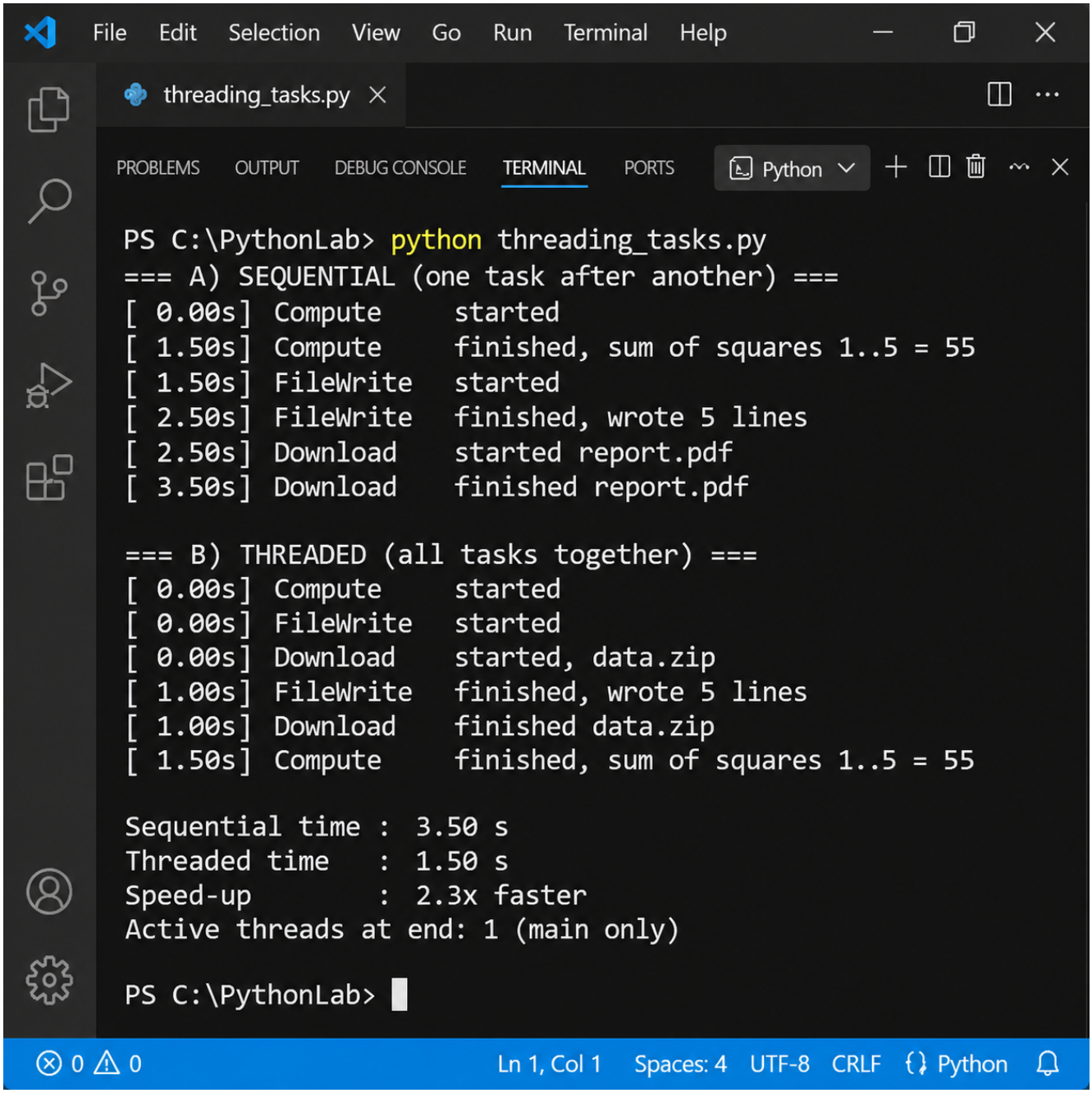
PYTHON MULTITHREADING PRACTICAL

Practical code and VS Code output

Name: Aditya Dhage **Class:** SY 12A **Roll No.:** 03

Aim: To run different tasks concurrently using multiple threads.

OUTPUT



The screenshot above shows the program output in Visual Studio Code.

SOURCE CODE

File name: threading.py

```
# Name       : Aditya Dhage
# Class      : SY 12A
# Roll No.   : 03
# Practical  : Multithreading in Python
# Aim        : To run different tasks concurrently using multiple threads.

import threading
import time
import tempfile
import os

# Shared data and lock
results = []
results_lock = threading.Lock()
START = time.perf_counter()


def record(task_name, message):
    """Safely add one line to the shared results list."""
    elapsed = time.perf_counter() - START
    with results_lock:
        results.append(f"[{elapsed:5.2f}s] {task_name:<10} {message}")
        print(results[-1])


# Task 1: Calculate the sum of squares
def compute_squares(n=5):
    record("Compute", "started")
    total = 0
    for i in range(1, n + 1):
        time.sleep(0.3)
        total += i * i
    record("Compute", f"finished, sum of squares 1..{n} = {total}")


# Task 2: Write lines into a text file
def write_file(path, lines=5):
    record("FileWrite", "started")
    with open(path, "w") as file:
        for i in range(1, lines + 1):
            file.write(f"line {i}\\n")
            time.sleep(0.2)
    record("FileWrite", f"finished, wrote {lines} lines")


# Task 3: Simulate a file download
def fake_download(name="report.pdf", seconds=1.0):
    record("Download", f"started {name}")
    time.sleep(seconds)
    record("Download", f"finished {name}")


# Run tasks one after another
def run_sequential(path):
    start_time = time.perf_counter()
    compute_squares()
    write_file(path)
    fake_download()
    return time.perf_counter() - start_time


# Run tasks concurrently using threads
def run_threaded(path):
    start_time = time.perf_counter()

    threads = [
        threading.Thread(target=compute_squares, name="T-Compute"),
        threading.Thread(target=write_file, args=(path,), name="T-File"),
        threading.Thread(
            target=fake_download,
            kwargs={"name": "data.zip"},
            name="T-Download"
        )
    ]
```

```
    for thread in threads:
        thread.start()

    for thread in threads:
        thread.join()

    return time.perf_counter() - start_time


if __name__ == "__main__":
    output_path = os.path.join(tempfile.gettempdir(), "thread_demo.txt")

    print("=== A) SEQUENTIAL (one task after another) ===")
    sequential_time = run_sequential(output_path)

    results.clear()
    START = time.perf_counter()

    print("\\n=== B) THREADED (all tasks together) ===")
    threaded_time = run_threaded(output_path)

    print(f"\\nSequential time : {sequential_time:.2f} s")
    print(f"Threaded time   : {threaded_time:.2f} s")
    print(f"Speed-up        : {sequential_time / threaded_time:.1f}x faster")
    print(f"Active threads at end: {threading.active_count()} (main only)")


# Expected result (times may vary slightly):
# Sequential time : approximately 3.50 s
# Threaded time   : approximately 1.50 s
# Speed-up        : approximately 2.3x faster
```